Hai detto:

cosa si intende per RPI bonds

ChatGPT ha detto:

Gli **RPI bonds** sono obbligazioni indicizzate all'inflazione basate sull'**RPI (Retail Price Index)**, un indice storico dell'inflazione nel Regno Unito.

In pratica:

- il **capitale** del bond viene rivalutato in base all'inflazione RPI;
- anche le **cedole** aumentano perché sono calcolate sul capitale rivalutato.

Questi titoli sono spesso chiamati anche **index-linked gilts** quando emessi dal governo britannico.

Come funzionano

Esempio semplice:

- compri un bond da £1.000 con cedola reale dell'1%;
- se l'RPI sale del 5%, il capitale "virtuale" diventa £1.050;
- la cedola dell'1% viene quindi calcolata su £1.050.

Risultato:

- protezione del potere d'acquisto;
- rendimento "reale" più stabile rispetto ai bond nominali.

Perché sono importanti

Gli investitori li usano per:

- proteggersi dall'inflazione;
- abbinare passività future indicizzate (fondi pensione, assicurazioni);
- diversificare rispetto ai bond tradizionali.

Attenzione però

L'RPI è un indice molto discusso perché:

- tende a mostrare un'inflazione più alta rispetto al CPI;
- il governo britannico sta progressivamente spostandosi verso misure come il CPIH.

Questo può influire sul valore futuro degli RPI-linked bonds.

Differenza rispetto ai bond normali

Bond normale	RPI bond
Cedola fissa	Cedola rivalutata con inflazione
Capitale fisso	Capitale indicizzato
Soffre inflazione alta	Protegge dall'inflazione
Più semplice	Più sensibile a tassi reali e aspettative inflazione

Se vuoi, posso anche spiegarti:

- ↳ come si valutano gli RPI bonds;
- ↳ differenza tra RPI, CPI e CPIH;
- ↳ rischi specifici degli index-linked gilts;
- ↳ perché i rendimenti reali possono essere negativi.

Hai detto:

come dovrei costruire, in java, lo schedule per questo tipo di bond?

ChatGPT ha detto:

Per costruire lo **schedule** di un RPI bond in Java devi separare due concetti:

1. **cashflow schedule nominale**
   (date cedole, fixing dates, maturity)
2. **meccanismo di indicizzazione inflattiva**
   (RPI lag, interpolation, index ratio)

La struttura tipica degli **UK index-linked gilts** è:

- cedola reale fissa
- capitale rivalutato con RPI
- inflation lag (tipicamente 3 mesi)
- spesso interpolazione giornaliera dell'indice

Architettura consigliata

Una buona modellazione è:

```
Bond
 ├── Schedule
 ├── InflationIndex
 ├── InflationLagRule
 ├── DayCount
 ├── CouponCalculator
 └── PrincipalRedemption
```

---

1. Schedule delle date

Lo schedule è normale:

```
public class CouponPeriod {
    LocalDate startDate;
    LocalDate endDate;
    LocalDate paymentDate;
}
```

Generator:

```
List<CouponPeriod> generateSchedule(
    LocalDate issueDate,
    LocalDate maturity,
    Frequency frequency
)
```

Per esempio semiannual:

```
15-Jan-2025
15-Jul-2025
15-Jan-2026
...
```

---

2. Inflation index time series

Serve una serie storica RPI:

```
public interface InflationIndex {
    double value(LocalDate date);
}
```

Implementazione semplice:

```
Map<YearMonth, Double> rpiFixings;
```

Esempio:

```
2025-01 -> 378.4
2025-02 -> 379.1
```

---

3. Inflation lag

Negli UK linker:

- payment date: 15-Jul-2026
- riferimento RPI: circa Apr-2026 (3 mesi lag)

Quindi:

```
LocalDate referenceDate =
    paymentDate.minusMonths(3);
```

---

4. Interpolazione

Molti linker UK usano interpolazione giornaliera.

Formula tipica:

```
RPI(t) =
RPI(M1) +
d / D * (RPI(M2) - RPI(M1))
```

dove:

- M1 = mese riferimento
- M2 = mese successivo
- d = giorno del mese
- D = giorni nel mese

In Java:

```
double interpolate(LocalDate date) {
```

```
    YearMonth m1 = YearMonth.from(date);
    YearMonth m2 = m1.plusMonths(1);

    double rpi1 = fixing(m1);
    double rpi2 = fixing(m2);

    int d = date.getDayOfMonth() - 1;
    int D = date.lengthOfMonth();

    return rpi1 + ((double)d / D) * (rpi2 - rpi1);
}
```

---

5. Index ratio

Elemento centrale.

```
IndexRatio =
ReferenceRPI / BaseRPI
```

dove:

- BaseRPI = indice all'emissione
- ReferenceRPI = indice interpolato del periodo

Java:

```
double indexRatio =
    referenceRpi / baseRpi;
```

---

6. Cedola inflation-linked

Cedola reale:

```
Coupon =
Notional
× IndexRatio
× RealCouponRate
× AccrualFactor
```

Java:

```
double coupon =
    notional *
    indexRatio *
    couponRate *
    accrualFactor;
```

---

7. Rimborso finale

```
Redemption =
Notional × FinalIndexRatio
```

Alcuni bond hanno:

- inflation floor
- principal floor (min 100%)

Quindi:

```
double redemption =
    notional *
    Math.max(finalIndexRatio, 1.0);
```

---

8. Modello completo consigliato

```
class InflationLinkedBond {

    double notional;
    double realCouponRate;

    Schedule schedule;

    InflationIndex rpiIndex;

    double baseRpi;

    int inflationLagMonths;

}
```

---

9. Oggetto cashflow

```
class Cashflow {

    LocalDate paymentDate;

    double couponAmount;

    double principalAmount;

    double indexRatio;
}
```

---

10. Design importante

NON salvare:

- cedole già calcolate

MA:

- calcolarle dinamicamente usando i fixing.

Perché:

- i fixing futuri cambiano
- pricing e risk richiedono recalculation

---

11. Caso reale: UK Gilts

Gli index-linked gilts hanno convenzioni specifiche:

- 3-month lag
- interpolazione lineare
- ex-dividend period
- real clean price
- dirty price inflation-adjusted

Conviene modellare:

`InflationConvention`

tipo:

```
enum InflationConvention {
    UK_IL_GILT,
    FR_OATI,
    US_TIPS
}
```

perché ogni mercato ha dettagli diversi.

---

12. Librerie utili

Se vuoi qualcosa di professionale:

- OpenGamma Strata
- QuantLib (C++ con binding Java)
- finmath lib

In particolare Strata ha:

- inflation curves
- price index values
- schedule generation
- day count
- linker pricing

molto vicini agli standard di mercato.

Se vuoi, posso anche mostrarti:

- ↳ una implementazione Java completa di schedule generator;
- ↳ come modellare i fixing RPI;
- ↳ pricing con curve reali;
- ↳ differenza fra clean/dirty price nei linker;
- ↳ esempio concreto di UK index-linked gilt.

Hai detto:

Mostrami una implementazione Java completa di schedule generator

ChatGPT ha detto:

```
import java.time.LocalDate;
import java.time.Month;
import java.time.Period;
import java.time.temporal.TemporalAdjusters;
import java.util.ArrayList;
import java.util.List;

/**
 * Complete schedule generator for bonds / inflation-linked bonds.
 *
 * Features:
 * - Monthly / Quarterly / Semiannual / Annual frequencies
 * - Forward generation
 * - Stub handling
 * - Business day adjustment
 * - End-of-month rule
 *
 * Pure Java 17+ implementation.
 */
public class BondScheduleGenerator {

    public static void main(String[] args) {

        LocalDate issueDate = LocalDate.of(2025, Month.JANUARY, 15);
        LocalDate maturityDate = LocalDate.of(2030, Month.JULY, 15);

        Schedule schedule = generateSchedule(
                issueDate,
                maturityDate,
                Frequency.SEMI_ANNUAL,
                BusinessDayConvention.MODIFIED_FOLLOWING,
                true
        );

        schedule.periods().forEach(System.out::println);
    }

    // ============================================================
    // PUBLIC API
```

```
// ============================================================

public static Schedule generateSchedule(
        LocalDate startDate,
        LocalDate endDate,
        Frequency frequency,
        BusinessDayConvention businessDayConvention,
        boolean endOfMonth
) {

    List<CouponPeriod> periods = new ArrayList<>();

    Period step = frequency.period();

    List<LocalDate> unadjustedDates = new ArrayList<>();

    LocalDate current = startDate;

    unadjustedDates.add(current);

    while (current.isBefore(endDate)) {

        current = current.plus(step);

        if (endOfMonth && isEndOfMonth(startDate)) {
            current = current.with(TemporalAdjusters.lastDayOfMonth());
        }

        if (current.isAfter(endDate)) {
            current = endDate;
        }

        unadjustedDates.add(current);

        if (current.equals(endDate)) {
            break;
        }
    }

    for (int i = 0; i < unadjustedDates.size() - 1; i++) {

        LocalDate unadjustedStart = unadjustedDates.get(i);
```

```
            LocalDate unadjustedEnd = unadjustedDates.get(i + 1);

            LocalDate adjustedStart =
                    adjustBusinessDay(unadjustedStart, businessDayConvention);

            LocalDate adjustedEnd =
                    adjustBusinessDay(unadjustedEnd, businessDayConvention);

            CouponPeriod period = new CouponPeriod(
                    adjustedStart,
                    adjustedEnd,
                    adjustedEnd,
                    unadjustedStart,
                    unadjustedEnd
            );

            periods.add(period);
        }

        return new Schedule(periods);
    }

    // ============================================================
    // BUSINESS DAY ADJUSTMENT
    // ============================================================

    public static LocalDate adjustBusinessDay(
            LocalDate date,
            BusinessDayConvention convention
    ) {

        if (isBusinessDay(date)) {
            return date;
        }

        return switch (convention) {

            case FOLLOWING ->
                    following(date);

            case MODIFIED_FOLLOWING ->
                    modifiedFollowing(date);
```

```
        case PRECEDING ->
                preceding(date);

        case UNADJUSTED ->
                date;
    };
}

private static LocalDate following(LocalDate date) {

    LocalDate d = date;

    while (!isBusinessDay(d)) {
        d = d.plusDays(1);
    }

    return d;
}

private static LocalDate preceding(LocalDate date) {

    LocalDate d = date;

    while (!isBusinessDay(d)) {
        d = d.minusDays(1);
    }

    return d;
}

private static LocalDate modifiedFollowing(LocalDate date) {

    LocalDate following = following(date);

    if (following.getMonth() != date.getMonth()) {
        return preceding(date);
    }

    return following;
}
```

```
// ============================================================
// CALENDAR
// ============================================================

/**
 * Simplified calendar:
 * weekends only.
 *
 * Real systems use TARGET / UK / NY calendars.
 */
public static boolean isBusinessDay(LocalDate date) {

    return switch (date.getDayOfWeek()) {

        case SATURDAY, SUNDAY -> false;

        default -> true;
    };
}

// ============================================================
// END OF MONTH
// ============================================================

private static boolean isEndOfMonth(LocalDate date) {

    return date.equals(
            date.with(TemporalAdjusters.lastDayOfMonth())
    );
}

// ============================================================
// DOMAIN OBJECTS
// ============================================================

public record Schedule(
        List<CouponPeriod> periods
) {
}

public record CouponPeriod(
```

```
        LocalDate startDate,
        LocalDate endDate,
        LocalDate paymentDate,

        LocalDate unadjustedStartDate,
        LocalDate unadjustedEndDate

) {

    @Override
    public String toString() {

        return "CouponPeriod{" +
                "start=" + startDate +
                ", end=" + endDate +
                ", payment=" + paymentDate +
                '}';
    }
}

// ============================================================
// ENUMS
// ============================================================

public enum Frequency {

    MONTHLY(Period.ofMonths(1)),
    QUARTERLY(Period.ofMonths(3)),
    SEMI_ANNUAL(Period.ofMonths(6)),
    ANNUAL(Period.ofYears(1));

    private final Period period;

    Frequency(Period period) {
        this.period = period;
    }

    public Period period() {
        return period;
    }
}
```

```
    public enum BusinessDayConvention {

        FOLLOWING,
        MODIFIED_FOLLOWING,
        PRECEDING,
        UNADJUSTED
    }
}
```

Questo genera uno schedule completo tipo:

```
CouponPeriod{start=2025-01-15, end=2025-07-15, payment=2025-07-15}
CouponPeriod{start=2025-07-15, end=2026-01-15, payment=2026-01-15}
CouponPeriod{start=2026-01-15, end=2026-07-15, payment=2026-07-15}
...
```

Per un RPI bond, sopra questo schedule aggiungeresti:

- inflation lag
- reference CPI/RPI dates
- interpolation
- index ratio
- inflation-adjusted cashflows

senza modificare il generator base.

Hai detto:

ma la paymentDate non coincide con la endDate del coupon period?

ChatGPT ha detto:

Nella maggior parte dei bond semplici **sì**, ma non sempre.
Per questo conviene tenerle separate nel modello.

Nel codice che ti ho mostrato:

`paymentDate = adjustedEnd`

quindi di fatto coincidono.

Perché separarle allora?

Perché nei mercati reali spesso:

```
accrualEndDate != paymentDate
```

Esempi tipici:

- pagamento T+2
- fixing delay
- inflation lag
- payment offset
- ex-coupon period
- holiday adjustment differente

Caso semplice

Bond vanilla:

```
Accrual Start : 15-Jan
Accrual End   : 15-Jul
Payment Date  : 15-Jul
```

Qui:

- endDate = paymentDate

Caso reale più corretto

Molti sistemi modellano:

```
Accrual End : 15-Jul
Payment     : 17-Jul
```

perché:

- il pagamento avviene 2 business days dopo.

Nei linker / inflation bonds

Può essere ancora più importante.

Esempio:

```
Coupon accrual end : 15-Jul
Reference RPI date : 15-Apr
Payment date       : 17-Jul
```

Quindi:

- il periodo economico finisce il 15-Jul
- il fixing inflattivo è laggato
- il cash viene pagato dopo

Modellazione professionale

Di solito si separano:

```
class CouponPeriod {

    LocalDate accrualStartDate;
    LocalDate accrualEndDate;

    LocalDate fixingDate;

    LocalDate paymentDate;
}
```

oppure:

```
class SchedulePeriod {

    LocalDate startDate;
    LocalDate endDate;
}

class Payment {

    LocalDate paymentDate;
}
```

Best practice

Per una libreria seria:

NON usare

```
endDate == paymentDate
```

come assunzione strutturale.

MEGLIO

tenerle distinte anche se inizialmente coincidono.

Versione migliore del generator

Per esempio:

```
LocalDate paymentDate =
    adjustBusinessDay(
        unadjustedEnd.plusDays(paymentLagDays),
        convention
    );
```

Così puoi supportare:

- T+0
- T+2
- T+5

senza rifare il modello.

Design consigliato per linker

```
record CouponPeriod(

    LocalDate accrualStart,
    LocalDate accrualEnd,

    LocalDate fixingDate,

    LocalDate paymentDate
```

)

dove:

```
fixingDate =
paymentDate - inflationLag
```

oppure:

- fixing lag in mesi
- interpolation reference date

Conclusione

Nel tuo esempio:

- sì, coincidono.

Ma nel modello:

- è corretto tenerle separate,
- soprattutto per inflation-linked bonds e sistemi market-grade.

Hai detto:

e quota interessi e capitale dove la trovo nel coupon?

ChatGPT ha detto:

Nel coupon period **non dovresti salvare direttamente quota interessi e quota capitale**.
Il `CouponPeriod` rappresenta solo il **periodo temporale** dello schedule.

Le componenti monetarie vanno calcolate in un livello successivo: i **cashflows**.

La separazione corretta è:

```
Schedule  -> solo date
Cashflow -> importi monetari
```

Struttura corretta

1. Schedule period

```
record CouponPeriod(

    LocalDate accrualStart,
    LocalDate accrualEnd,
    LocalDate paymentDate

) {}
```

Qui NON ci sono soldi.

---

2. Cashflow

Poi generi:

```
record Cashflow(

    LocalDate paymentDate,

    double interestAmount,

    double principalAmount,

    double totalAmount

) {}
```

Perché è importante separare?

Perché:

Lo schedule è statico

Le date:

- non cambiano
- non dipendono dal pricing
- non dipendono da curve o fixing

I cashflow sono dinamici

Dipendono da:

- fixing RPI
- notional residuo
- inflation ratio
- amortization
- forecast curve
- default assumptions

Bond tradizionale

Esempio:

```
Nominale: 1000
Cedola: 4%
Frequenza: semestrale
```

Periodo:

```
15-Jan -> 15-Jul
```

Interesse:

```
1000 × 4% × 0.5 = 20
```

Capitale:

```
0
```

Alla maturity:

```
Interest = 20
Principal = 1000
```

Quindi il coupon NON contiene il capitale

Normalmente:

```
Coupon = interesse
Redemption = capitale
```

Ma nei linker cambia un po’

Negli inflation-linked bonds:

```
Interest =
RealCoupon × InflationAdjustedPrincipal
```

Quindi:

- il capitale rivalutato entra nel calcolo interessi
- ma il rimborso capitale resta separato

Esempio linker

```
Nominale originale = 1000
Index Ratio = 1.18
Real coupon = 1%
Semiannual
```

Capitale indicizzato:

```
1000 × 1.18 = 1180
```

Cedola:

```
1180 × 1% × 0.5 = 5.90
```

Rimborso finale:

```
1180
```

Modellazione professionale

Molto meglio così:

```
sealed interface Cashflow
    permits CouponCashflow,
            RedemptionCashflow {
}
```

Cedola

```
record CouponCashflow(

    LocalDate paymentDate,

    double accrualFactor,

    double rate,

    double notional,

    double amount

) implements Cashflow {}
```

Rimborso capitale

```
record RedemptionCashflow(

    LocalDate paymentDate,

    double principalAmount

) implements Cashflow {}
```

Per linker professionale

Aggiungi:

```
record InflationCouponCashflow(

    LocalDate paymentDate,

    double realRate,

    double indexRatio,

    double adjustedNotional,

    double amount

) implements Cashflow {}
```

Architettura corretta

```
BondDefinition
    ↓

ScheduleGenerator
    ↓

CouponPeriods
    ↓

CashflowGenerator
    ↓

Cashflows
```

Errore comune

Molti fanno:

```
class CouponPeriod {

   LocalDate start;
   LocalDate end;

   double coupon;
   double principal;
}
```

Problema:

- mescoli calendario e pricing
- difficile supportare forecast/repricing
- difficile fare risk
- difficile supportare inflation fixing futuri

Design market-grade

Separare:

date logic

da

valuation logic

è fondamentale.

È esattamente come fanno:

- OpenGamma Strata
- QuantLib
- Murex
- Calypso Technology

Se vuoi, posso anche mostrarti:

- ↳ una vera cashflow engine per linker;
- ↳ come generare coupon + redemption automaticamente;
- ↳ come modellare amortizing bonds;
- ↳ come calcolare accrued interest;
- ↳ clean price vs dirty price nei linker;

- ↳ design stile QuantLib/Strata.

Hai detto:

Mostrami design stile QuantLib/Strata

ChatGPT ha detto:

Il design stile QuantLib / OpenGamma Strata è fortemente basato su questi principi:

```
Trade
  ↓
Product Definition
  ↓
Resolved Product
  ↓
Schedule
  ↓
Cashflows
  ↓
Pricer
```

L'idea chiave è:

- gli oggetti "definition" sono IMMUTABILI e leggeri
- i cashflow vengono derivati
- il pricing non vive dentro il bond
- separazione totale fra:
    - calendario
    - fixing
    - curve
    - pricing

---

1. Product Definition

Questo è il bond "contrattuale".

```
public final class InflationLinkedBond {

    private final double notional;
```

```
    private final double realCoupon;

    private final LocalDate issueDate;

    private final LocalDate maturityDate;

    private final Frequency frequency;

    private final DayCount dayCount;

    private final BusinessDayConvention businessDayConvention;

    private final PriceIndex priceIndex;

    private final int inflationLagMonths;

    private final double baseIndex;

    // constructor + getters
}
```

NOTA:

- nessun cashflow
- nessun prezzo
- nessun fixing calcolato

Solo termini contrattuali.

---

2. Schedule

Separato completamente.

```
public final class SchedulePeriod {

    private final LocalDate startDate;

    private final LocalDate endDate;

    private final LocalDate paymentDate;
```

```
    // constructor
}
```

Generator:

```
public interface ScheduleGenerator {

    List<SchedulePeriod> generate(
        InflationLinkedBond bond
    );
}
```

---

3. Resolved Product

Questo è il concetto più importante di Strata.

La “resolved” version:

- espande tutte le convenzioni
- contiene schedule concreti
- pronta per pricing

```
public final class ResolvedInflationLinkedBond {

    private final InflationLinkedBond product;

    private final List<ResolvedCapitalIndexedCouponPeriod> couponPeriods;

    private final ResolvedRedemption redemption;

}
```

---

4. Resolved Coupon Period

Qui iniziano i dettagli market-grade.

```
public final class ResolvedCapitalIndexedCouponPeriod {

    private final LocalDate accrualStartDate;
```

```
    private final LocalDate accrualEndDate;

    private final LocalDate paymentDate;

    private final double accrualFactor;

    private final double realCoupon;

    private final double notional;

    private final PriceIndexObservation startObservation;

    private final PriceIndexObservation endObservation;

}
```

---

5. PriceIndexObservation

Fondamentale nei linker.

```
public final class PriceIndexObservation {

    private final PriceIndex index;

    private final YearMonth fixingMonth;

}
```

Per esempio:

```
Coupon payment: Jul-2028
Lag: 3M
Observation: Apr-2028
```

---

6. Inflation calculation separata

NON dentro il coupon.

```
public interface InflationRateComputation {

    double computeIndexRatio(
        PriceIndexValues values
    );
}
```

---

7. PriceIndexValues

Oggetto che fornisce fixing.

```
public interface PriceIndexValues {

    double value(YearMonth month);

}
```

Implementazioni:

- historical fixings
- projected curve
- market data provider

---

8. Cashflow derivation

Il coupon NON contiene amount precomputato.

Il pricer lo calcola.

```
public final class InflationCouponPricer {

    public double presentValue(
        ResolvedCapitalIndexedCouponPeriod period,
        RatesProvider provider
    ) {

        double indexRatio =
            computeIndexRatio(period, provider);

        double coupon =
```

```
            period.getNotional()
            * indexRatio
            * period.getRealCoupon()
            * period.getAccrualFactor();

        double df =
            provider.discountFactor(
                period.getPaymentDate()
            );

        return coupon * df;
    }
}
```

---

9. RatesProvider

Pattern centrale di Strata.

```
public interface RatesProvider {

    DiscountFactors discountFactors();

    PriceIndexValues priceIndexValues(
        PriceIndex index
    );
}
```

Questo permette:

- scenario analysis
- stress
- historical replay
- multi-curve

senza toccare il bond.

---

10. Redemption separata

Molto importante.

```
public final class ResolvedRedemption {

    private final LocalDate paymentDate;

    private final double notional;

    private final PriceIndexObservation observation;

}
```

Perché:

- principal repayment
- coupon
- inflation adjustment

sono concetti distinti.

---

11. DayCount separato

```
public interface DayCount {

    double yearFraction(
        LocalDate start,
        LocalDate end
    );
}
```

Implementazioni:

- ACT/ACT ICMA
- ACT/365F
- 30E/360

---

12. Calendars separati

```
public interface HolidayCalendar {

    boolean isBusinessDay(LocalDate date);
```

```
    LocalDate next(LocalDate date);
}
```

---

13. BusinessDayAdjustment

```
public final class BusinessDayAdjustment {

    private final BusinessDayConvention convention;

    private final HolidayCalendar calendar;

}
```

---

14. Risultato finale

Il flusso completo:

```
InflationLinkedBond
        ↓ resolve()
ResolvedInflationLinkedBond
        ↓
CouponPeriods
        ↓
Pricer
        ↓
PV / Risk / Sensitivities
```

---

15. Perché questo design è potente

Perché separa:

Concetto	Responsabilità
Bond	contratto
Schedule	date

Concetto	Responsabilità
Observation	fixing
Market data	curve/fixing
Pricer	matematica
Cashflow	risultato monetario

---

16. Errore tipico dei junior

Mettere:

```
bond.price()
bond.calculateCoupon()
bond.discount()
```

DENTRO il bond.

QuantLib/Strata NON fanno così.

Il bond è:

- immutable
- dumb
- declarative

Il pricing è esterno.

---

17. Versione minimal production-grade

Per un sistema serio ti servono almeno:

```
domain/
  Bond
  CouponPeriod
  Schedule

market/
```

```
  Curve
  Fixings
  RatesProvider

pricing/
  BondPricer
  DiscountingPricer
  InflationPricer

convention/
  DayCount
  Calendar
  BusinessDayConvention

resolution/
  ResolvedBond
```

---

18. Design pattern usati

QuantLib/Strata usano molto:

- Immutable objects
- Strategy pattern
- Factory pattern
- Resolver pattern
- Provider pattern
- Composition over inheritance

e pochissima ereditarietà profonda.

---

19. Architettura ideale moderna Java

Con Java 21:

```
record
sealed interface
immutable collections
```

sono perfetti per financial engineering.

20. Conclusione

Il punto centrale del design market-grade è:

`Instrument ≠ Pricing`

e:

`Schedule ≠ Cashflow`

Questa separazione è ciò che rende:

- scalabile
- testabile
- estendibile
- supportabile per decine di strumenti diversi.

Hai detto:

casa e`una sealed interface

ChatGPT ha detto:

Una **sealed interface** in Java è un'interfaccia che può essere implementata **solo da un insieme limitato e dichiarato di classi**.

È stata introdotta in Java 17.

Serve per modellare gerarchie "chiuse" e controllate, molto utili in:

- financial engineering
- AST/parser
- algebraic data types
- event systems
- domain modeling

Problema delle interfacce normali

Con una normale interface:

```
interface Cashflow {}
```

chiunque può fare:

```
class WeirdCashflow implements Cashflow {}
```

anche in un altro modulo/libreria.

Per sistemi finanziari questo è spesso pericoloso:

- perdi controllo del dominio
- switch incompleti
- casi non previsti

Con sealed interface

Puoi dire:

```
public sealed interface Cashflow
    permits CouponCashflow,
            RedemptionCashflow {
}
```

Ora SOLO queste classi possono implementarla:

```
public final class CouponCashflow
    implements Cashflow {
}
```

```
public final class RedemptionCashflow
    implements Cashflow {
}
```

Vantaggio enorme

Il compilatore conosce TUTTI i tipi possibili.

Quindi puoi fare:

```
double amount(Cashflow cf) {

    return switch (cf) {
```

```
        case CouponCashflow c ->
            c.amount();

        case RedemptionCashflow r ->
            r.principal();
    };
}
```

senza `default` .

Perché:

- la gerarchia è chiusa
- il compilatore sa che non esistono altri tipi

Questo è potentissimo nei bond

Per esempio:

```
sealed interface PaymentEvent
```

con:

- coupon
- redemption
- fee
- inflation adjustment
- amortization

Esempio completo

```
public sealed interface Cashflow
    permits CouponCashflow,
            RedemptionCashflow {

    LocalDate paymentDate();
}
```

Coupon

```
public record CouponCashflow(

    LocalDate paymentDate,

    double amount,

    double accrualFactor

) implements Cashflow {
}
```

Redemption

```
public record RedemptionCashflow(

    LocalDate paymentDate,

    double principal

) implements Cashflow {
}
```

Uso

```
List<Cashflow> cashflows = List.of(
    new CouponCashflow(
        LocalDate.of(2026, 7, 15),
        12.5,
        0.5
    ),
    new RedemptionCashflow(
        LocalDate.of(2030, 7, 15),
        1000
    )
);
```

Pattern matching moderno

Con Java moderno:

```
for (Cashflow cf : cashflows) {

    switch (cf) {

        case CouponCashflow c ->
            System.out.println(
                "Coupon: " + c.amount()
            );

        case RedemptionCashflow r ->
            System.out.println(
                "Principal: " + r.principal()
            );
    }
}
```

Differenza con abstract class

Una `sealed interface` :

- permette multiple inheritance of type
- più flessibile
- più “functional/domain oriented”

Una `sealed abstract class` :

- può avere stato condiviso
- behavior comune

Altri modificatori collegati

Le classi che implementano una sealed interface devono essere:

final

nessuno può estenderle

```
final class CouponCashflow
```

---

sealed

può essere ulteriormente limitata

```
sealed class CouponCashflow
    permits FixedCoupon, FloatingCoupon
```

---

non-sealed

riapre la gerarchia

```
non-sealed class ExoticCashflow
```

Esempio finanziario realistico

```
sealed interface RateComputation
```

implementazioni:

```
FixedRateComputation
IborRateComputation
OvernightCompoundedRateComputation
InflationRateComputation
```

Questo è molto simile a come ragiona Strata.

Perché sono ottime nel finance

Perché gli strumenti finanziari sono:

- fortemente tipizzati

- con domini chiusi
- pieni di varianti note

Perfetto per sealed types.

Design moderno consigliato

Per financial libraries Java moderne:

usa:

- `record`
- `sealed interface`
- `enum`
- immutabilità

evita:

- inheritance profonda
- mutabilità
- mega-classi God Object

In pratica

Per il tuo caso:

```
sealed interface Cashflow
```

è molto meglio di:

```
abstract class Cashflow
```

perché:

- più sicuro
- più espressivo
- migliore pattern matching
- più vicino al domain modeling moderno.

Hai detto:

come faccio a inizializzare una LocalDate ad es al 1/11/1998?

ChatGPT ha detto:

In Java usi `LocalDate.of(...)` .

Per il 1 novembre 1998:

```
LocalDate date = LocalDate.of(1998, 11, 1);
```

oppure usando `Month` :

```
LocalDate date = LocalDate.of(1998, Month.NOVEMBER, 1);
```

Ricorda:

- ordine = `anno, mese, giorno`
- `LocalDate` fa parte di `java.time`

Quindi:

```
import java.time.LocalDate;
import java.time.Month;
```

Esempio completo:

```
import java.time.LocalDate;
import java.time.Month;

public class Main {

    public static void main(String[] args) {

        LocalDate date =
            LocalDate.of(1998, Month.NOVEMBER, 1);

        System.out.println(date);
    }
}
```

Output:

```
1998-11-01
```

Hai detto:

come ottengo una java.sql.date da una LocalDate?

ChatGPT ha detto:

Puoi convertire una `LocalDate` in `java.sql.Date` usando:

```
java.sql.Date sqlDate =
    java.sql.Date.valueOf(localDate);
```

Esempio completo:

```
import java.time.LocalDate;

public class Main {

    public static void main(String[] args) {

        LocalDate localDate =
            LocalDate.of(1998, 11, 1);

        java.sql.Date sqlDate =
            java.sql.Date.valueOf(localDate);

        System.out.println(sqlDate);
    }
}
```

Output:

```
1998-11-01
```

Conversione inversa

Da `java.sql.Date` a `LocalDate`:

```
LocalDate localDate =
    sqlDate.toLocalDate();
```

Nota importante

Nelle applicazioni Java moderne:

- usa `LocalDate` nel dominio
- usa `java.sql.Date` solo al boundary JDBC/database

`java.sql.Date` è legacy e deriva da `java.util.Date` .